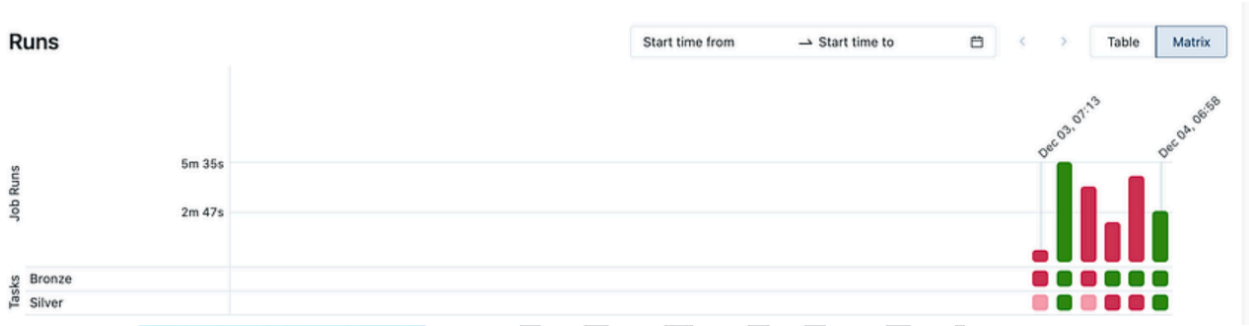
🚀 **Data Engineering Daily – Day 22**

📄 Page 1/4: The Challenge — Failing Streaming Job



❌ **Scenario:**
A PySpark Structured Streaming job consuming **Kafka events (100M+/day)** keeps failing due to **executor crashes**.

When restarted, it **reprocesses old data → duplicates in Delta table**.

What the interviewer wants to know:

- Can you explain **why checkpointing matters**?
- Can you apply **fault-tolerance techniques** (not just theory)?
- Can you balance **low latency vs. high reliability**?

DAILY DATA ENGINEERING BY MAYANK

👉 **Common Issues in Streaming Jobs:**

- No checkpointing → duplicates
- Improper state management → aggregations lost
- Broadcasting big tables → OOM failures

Swipe → Let's look at an optimization playbook.

📄 Page 2/4: The Playbook (with Code)

✅ Enable Checkpointing

```
stream_df = (
    spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "server:9092")
    .option("subscribe", "transactions")
    .load()
)
query = (
    stream_df.writeStream
    .format("delta")
    .option("checkpointLocation", "dbfs:/checkpoints/txn_pipeline/")
    .outputMode("append")
    .start("dbfs:/delta/transactions")
)
```

DAILY DATA ENGINEERING BY MAYANK

✅ Idempotent Writes with Delta Merge

```
from delta.tables import DeltaTable
target = DeltaTable.forPath(spark, "dbfs:/delta/transactions")
updates = ...  # streaming DataFrame
(target.alias("t")
 .merge(updates.alias("u"), "t.txn_id = u.txn_id")
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())
```

📄 Page 3/4: Real-World Example



🔹 **Context:**
We had a **financial streaming pipeline** in Databricks processing **100M+ Kafka events/day** into Delta Lake.

🔹 **The Problem:**

- A cluster node failed during ingestion.
- On restart, Spark **reprocessed old Kafka offsets**, leading to **duplicate transactions** in the Delta table.
- In finance, this meant **wrong balances and reports** — a critical issue.

🔹 **The Solution:**
✅ Enabled **checkpointing in ADLS** so Spark could resume from the exact last processed offset.
✅ Used **Delta Merge (UPSERT)** instead of simple append → ensured transactions were written only once.
✅ Configured **exactly-once semantics** → no duplication even after failures.

📊 **Impact:**

- **Duplicates eliminated**
- **Recovery time cut from hours to minutes**
- **Pipeline became production-grade reliable**

DAILY DATA ENGINEERING BY MAYANK

📄 Page 4/4: Key Takeaways

✨ **Always enable checkpointing** in streaming jobs → without it, restarts = duplicates or data loss.

✨ Use **idempotent writes (Delta Merge/UPSERT)** instead of blind appends to guarantee data correctness.

✨ Balance **latency vs. reliability** → shorter trigger intervals for real-time needs, micro-batches for stable pipelines.

✨ **Design for failures upfront** → Spark clusters WILL fail. Build fault tolerance & recovery strategy before production.

⚖️ **Interview Tip:**
Don't just say *"restart the job"*. Show that you understand **how Spark Streaming ensures fault tolerance**.

👉 **Final Answer (Interview Style):**
"In one project, my streaming job from Kafka to Delta was duplicating records after cluster restarts. I solved this by enabling checkpointing in ADLS to track offsets, and implemented Delta Merge for idempotent writes. This ensured exactly-once delivery, eliminated duplicates, and reduced recovery time to minutes."

💡 **Tomorrow (Day 23):** "How to Design a Fault-Tolerant Data Pipeline in ADF."

DAILY DATA ENGINEERING BY MAYANK